

Supplementary material

Blockchain access control for agricultural edge IoT: a 61-day Hyperledger Fabric field study

Forcha Glen Beloa Michael Ekonde Sone Elie Fute Tagne
André Paul Mvogo Asse

Algorithms S1 to S3 give protocol detail referenced from the main article: cascading enrollment, residue encoding with two-of-three recovery, and revocation publication with cache invalidation. Section S2 records two operational observations from the deployment that no performance metric captures. Every number quoted here expands from the same generated macro file the main article uses, so both regenerate together from `analysis/derive_results.py` in the released artifact.

S1 Protocol algorithms and diagrams

Input: sensor s with declared zone z_s , gateway g with zone z_g
Output: zone-bound certificate in RTC memory, private key written to eFuse
On the sensor:
 $(sk, pk) \leftarrow \text{ED25519KEYGEN}()$
 $\text{EFUSEWRITE}(\text{BLK3}, sk)$ ▷ one-time programmable, survives reflash
 $csr \leftarrow \text{MAKECSR}(pk, z_s); \sigma \leftarrow \text{SIGN}_{sk}(csr)$
send (csr, σ) to g over LoRa
On the gateway:
if $z_s \neq z_g$ **then**

return reject ▷ zone binding enforced at the RA
end if
if $\neg \text{RATELIMITOK}(src)$ **then**

return reject
end if
if $\neg \text{VERIFYSIG}(csr, \sigma, pk)$ **then**

return reject
end if
 $cert \leftarrow \text{CAISSUE}(csr, \text{SAN}=z_g, \text{NOTAFTER}=t+6 \text{ months})$
return $cert$ to s
On the sensor, each wake:
 $cert \leftarrow \text{RTCREAD}()$
if $\neg \text{MAGICOK}() \vee \neg \text{CRCOK}(cert)$ **then**
restart enrollment ▷ never transmit under unverified identity
end if
 $t \leftarrow \text{GPSTIME}();$ **if** unavailable **then** $t \leftarrow \text{NTPTIME}()$
if $t \text{ available} \wedge cert.\text{NOTAFTER} < t$ **then**
restart enrollment
end if

Algorithm S1: Stage 3: sensor enrollment through the zone gateway

Input: reading v with $0 \leq v < M_2$, moduli $m_1=97, m_2=101, m_3=103$; $M_2 = \min_{i \neq j} m_i m_j = 97 \cdot 101 = 9797$

Output: v recovered at the gateway from any two received residues

Encode (sensor):

for $i \in \{1, 2, 3\}$ **do**

$r_i \leftarrow v \bmod m_i$

▷ one byte each

transmit packet $\langle i, r_i \rangle$ at reduced TX power

end for

Recover (gateway):

$S \leftarrow$ set of successfully received $\langle i, r_i \rangle$

if $|S| < 2$ **then**

return $\perp$

▷ unrecoverable, reading dropped

end if

pick any two indices $\{j, k\} \subseteq S$; $M' \leftarrow m_j m_k$

▷ $v < M_2 \leq M'$ guarantees uniqueness

$M_j \leftarrow M'/m_j$; $M_k \leftarrow M'/m_k$

$y_j \leftarrow M_j^{-1} \bmod m_j$; $y_k \leftarrow M_k^{-1} \bmod m_k$

$v' \leftarrow (r_j M_j y_j + r_k M_k y_k) \bmod M'$

if $|S| = 3$ **then**

verify $v' \equiv r_l \pmod{m_l}$ for the third index l

▷ integrity cross-check

end if

return v'

Algorithm S2: Residue encoding and two-of-three recovery

Input: certificate C to revoke, reason ρ , peer set P , gateway set G

Output: C rejected network-wide; staleness bounded and recorded

CAAPPENDCRL(C, ρ); $crl \leftarrow$ PUBLISHCRL()

▷ measured: 60.4s mean

LEDGERWRITE(REVOCATIONEVENT, C, ρ, now)

for all $p \in P$ **do**

gossip crl to p

▷ measured: 4.21 min to reach all peers

end for

for all $g \in G$ **do**

if REACHABLE(g) **then**

INVALIDATEPOLICYCACHE(g)

▷ staleness bounded by gossip, not by TTL

else

mark event DELAYED; retry on reconnect

▷ observed for 29.5% of events

staleness bounded by scheduled CRL refresh, up to 24h

end if

end for

Algorithm S3: Revocation publication and cache invalidation

S2 Operational and measurement notes

S2.1 Operational observations

Two aspects of running the system for two months do not show up in any performance metric.

Zone certificate binding solved an administrative problem it was not designed for. Worker assignments spanning cooperating farms, which would otherwise need separately maintained access-control configuration, resolved through the zone attribute already embedded in device and user certificates for operational reasons. Temporal role expiry removed another class of manual work, with seasonal credentials lapsing on schedule without administrator intervention or database editing, each lapse leaving an audit record rather than a silent deletion.

The enrollment protocol needed three firmware iterations before it survived multi-hour gateway outages, and the recurring failure mode was time. A sensor waking with an unsynchronised clock cannot evaluate certificate expiry, and early firmware either refused to transmit or transmitted under an identity it could not validate. The fix appears as the final block of Supplement S1: acquire time from GPS first with NTP as fallback, and skip expiry enforcement for a single cycle rather than strand the node when neither is available. A testbed that never loses its time source cannot surface this class of fault.

S2.2 Energy-measurement scope

The main article’s “Transmission energy and cryptographic cost” section reports only the headline reduction in per-event transmission energy between single-channel and residue-coded LoRa modes. The full measurement: per-event energy falls from 24.20 mJ (SD 0.82) to 12.41 mJ (SD 0.41), a reduction of 48.7% [48.6, 48.8]; per-packet airtime falls from 36.1 ms to 31.5 ms; and the instrumented packets average 20.5 B and 10.5 B respectively. Cryptographic cost on the sensor is negligible at this duty cycle: the signing pipeline takes 900 μ s (SD 9), comprising Ed25519 signing at 860 μ s, hardware SHA-256 at 12.0 μ s, eFuse key read at 8.0 μ s and context management at 20.0 μ s, or 0.90 ms per wake against the 30-minute cycle. Three qualifications bound what the energy reduction describes, given here in full since the main text states only their conclusion.

First, the instrumented payloads are smaller than a complete authenticated frame: an Ed25519 signature alone is 64 B before nonce, timestamp and identity fields are added, so the measured percentage is a reduction on the instrumented payload, not on the frame a corrected, fully authenticated deployment would actually transmit. Second, the energy and airtime samples come from repeated measurement of one instrumented sensor node on the bench, so the reported interval reflects measurement repeatability on that node rather than variation across the fifty deployed devices; a multi-device energy campaign, ideally over the corrected full frame, would be needed to generalise the reduction across the fleet. Third, at the deployed 30-minute duty cycle, transmission energy is a small fraction of a node’s daily budget against deep-sleep consumption, so even the full measured reduction moves estimated battery life by roughly one percent; the value of the measurement is that it is cleanly attributable to the encoding choice, not that it materially extends field battery life. We report the reduction because it is cleanly measured and make no stronger battery-life claim than that.

S2.3 Signature test vector

A fixed, reproducible vector for the firmware’s sign path (`esp32/main/signing.c`: SHA-256 the packed struct, then Ed25519-sign the 32-byte digest), so an independent reimplement in any language can check byte-for-byte agreement without running this repository’s code. Field order for the packed struct is little-endian `device_id` (u32), `reading_id` (u16), `soil_raw` (u16), `temperature_centi_c` (i16), `timestamp` (i64); the example uses device id 42, reading id 7,

raw soil 2048, temperature 25.00 °C (centi-degrees 2500) and timestamp 1735689600 (2025-01-01T00:00:00Z). The private key is the Ed25519 seed 00 01 02 ... 1e 1f (32 bytes, byte i has value i), used only to make this vector reproducible and not a key from any deployed device.

```
seed:      000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
pubkey:    03a107bff3ce10be1d70dd18e74bc09967e4d6309ba50d5f1ddc8664125531b8
payload:   2a00000007000008c4098085746700000000
digest:    29cf4c16f38f0a5bc8cef0da69ed7c9e4d570579fcd2d347a8d063895b49599a
sig:       5c59e0f3fef48a23c09ae90c1ac638d179e1058569f2469323f43907b4ab8c4
           d701931da85a762a019cda7174e5a6e418326990f5b51207119e7bd83f124fc08
```

Ed25519-verifying `sig` against `digest` with `pubkey` must succeed; verifying it against the SHA-256 digest of `payload` with any single byte flipped must fail. `gateway/tests/test_signature_binding.py` (`test_canonical_vector_matches_firmware_format` and `test_canonical_vector_rejects_single_byte_mod`) checks both directions and is the executable form of this vector.

Table S1: Evaluation methods reported by representative blockchain access-control studies.

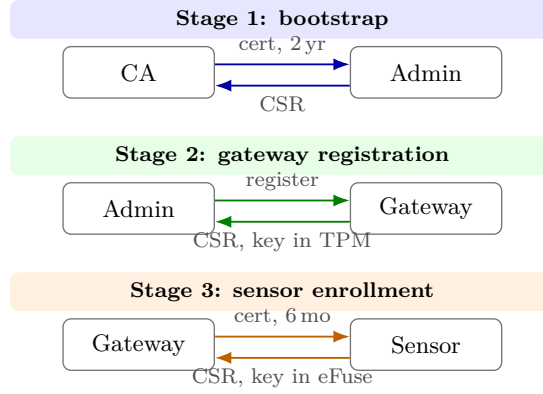
System	Platform	Setting	Duration	Auth. span timed	Revocation measured
FairAccess [5]	Bitcoin	Analytical	N/A	No	No
BlendCAC [10]	Ethereum	Testbed	Hours	No	No
Novo [4]	Ethereum	Testbed	Hours	No	No
Shih et al. [8]	Fabric	Lab (ARM64)	Days	No	No
Zaidi et al. [11]	Fabric	Simulation	N/A	No	No
Pajoooh et al. [6]	Fabric	Lab	N/R	No	No
Putra et al. [7]	Fabric	Lab	N/R	No	No
Usman et al. [9]	Fabric	Cloud	Days	No	No
González et al. [2]	Fabric	Lab	N/R	No	No
Idrissi & Palmieri [3]	Fabric	Lab (ARM)	N/R	Partial	No
Bandara et al. [1]	Fabric	Simulation	N/A	No	No
This work	Fabric	Field	61 days	Yes	Yes

S3 Related-system comparison

Table S1 summarises how the systems discussed in the main article’s Related work section are evaluated, extending the discussion there to the full comparison set.

Table S2: Effective permissions generated from the deployed chaincode.

Permission	Sensor	Gateway	Farmer	Agronomist	Certifier	Supply	Admin
Read own	:	:	✓	✓	✓	✓	✓
Read zone	:	✓	✓	✓	✓	:	✓
Read all	:	:	:	:	:	:	✓
Write sensor	✓	✓	✓	:	:	:	✓
Control zone	:	✓	✓	✓	:	:	✓
Control all	:	:	:	:	:	:	✓
Read audit	:	:	✓	✓	✓	:	✓
Issue CZT	:	:	✓	✓	:	:	✓
Manage roles	:	:	:	:	:	:	✓
Admin all	:	:	:	:	:	:	✓



Registration authority moves one tier further from the CA at each stage. The sensor never runs a TLS handshake, and its private key is generated on-device and never transmitted. It is not read-protected: see the main article’s “Cascading enrollment” section.

Figure S1: Three-stage cascading enrollment.

S4 Full permission matrix and protocol diagrams

The main article’s compact permission-difference table (“Access-control design”) gives only the rows that differ between the intended, deployed and corrected policies. Table S2 is the complete effective-permission matrix for all seven roles, generated directly from the deployed chaincode (`roles.go`) rather than transcribed by hand, so it cannot silently drift from the code that was measured.

Figure S1 illustrates the three-stage cascading enrollment protocol whose pseudocode is Algorithm S1, and Figure S2 places the access-control check inside Fabric’s execute-order-validate pipeline, referenced from the main article’s Implementation section.

Algorithm S4 states the access-decision procedure the main article’s Equation (3) describes in full (moved here from the main text to save space; two of its properties are summarised in the main article’s “Zone and temporal scoping” section).

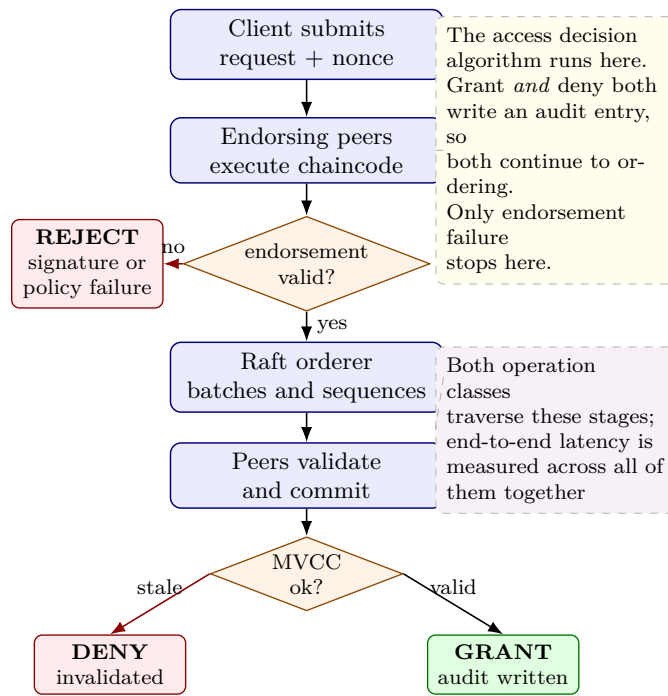


Figure S2: Access control within the Fabric execute-order-validate transaction flow.

Input: request (u, op, res, t) , caller certificate C

Output: GRANT or DENY. Every path writes an audit entry, so every invocation mutates state and commits through ordering.

```

 $n \leftarrow \text{NONCE}(req)$ 
if SEENBEFORE( $n$ ) then
  AUDIT(DENY, REPLAYDETECTED);
return DENY
end if
 $z_u \leftarrow \text{PARSEZONEASN1}(C)$  ▷ DER bytes, not PEM text
if ONCRL( $C$ ) then
  AUDIT(DENY, CERTREVOKED);
return DENY
end if
 $A \leftarrow \text{ROLEASSIGNMENTS}(u)$ 
 $R_{\text{act}} \leftarrow \emptyset$ 
for  $a \in A$  do
  if  $a.\text{ExpiresAt} < t$  then
    REVOKEROLE( $a$ ) ▷ state-mutating: forces endorsement
  else
     $R_{\text{act}} \leftarrow R_{\text{act}} \cup \{a.\text{role}\}$ 
  end if
end for
if  $R_{\text{act}} = \emptyset$  then
  AUDIT(DENY, ROLEINSUFFICIENT);
return DENY
end if
 $P \leftarrow \bigcup_{r \in R_{\text{act}}} \bigcup_{r' \preceq r} pa(r')$  ▷ inheritance closure, depth  $\leq 3$ 
if  $(op, res) \notin P$  then
  AUDIT(DENY, OPNOTPERMITTED);
return DENY
end if
 $z_r \leftarrow \text{ZONEOF}(res)$ 
if  $z_u \neq z_r \wedge \neg \text{VALIDCZT}(u, z_r, t)$  then
  AUDIT(DENY, ZONEMISMATCH);
return DENY
end if
RECORDNONCE( $n$ ); AUDIT(GRANT,  $(op, res, z_r)$ )

return GRANT

```

Algorithm S4: Access decision (**CheckAccess**)

Table S3: Topology and routing used for each peer-scaling configuration.

Parameter	Value
Configured logical peers	4, 8, 16, 32
Physical hosts	4 (Raspberry Pi 4B, one per zone)
Peers per host	1, 2, 4, 8 respectively
Endorsement policy	<code>OR('FarmMSP.member')</code> , one endorsement
Endorsements required	1 per transaction, any peer
Proposal selection	Fabric Gateway service default (discovery-based); not application-controlled
Load balancing	Delegated to Gateway service discovery; the client issues one submission per transaction
State database	CouchDB 3.2, one instance per peer
Block cut	<code>BatchTimeout</code> 2s, max 10 messages, 512kB preferred size
Orderer topology	3 Raft orderers, $f=1$, unchanged across rows
CPU / memory allocation	No cgroup limits set; peers above 4 share the host of the corresponding zone's deployed peer

Table S4: Experimental unit used for each analysis in the main article.

Analysis	Independent unit
Throughput vs. concurrency	Independent, interleaved benchmark run (5 per cell per arm)
Access-control invariance across peer counts	Independent run, 5 per configuration, counterbalanced order
Peer-count latency comparison	Independent run, 5 per configuration, counterbalanced order
Operation-label diagnostic	Deployment day (61 days)
Revocation stage records	Reported descriptively; stages do not resolve into independent episodes
Security campaign	Executed test corpus, not a random sample of attacks; no interval given

S5 Experimental design detail

These tables give the full experimental-design detail behind the main article's Measurement methodology section: the peer-scaling topology, the independent experimental unit used for every comparison in the article, and (in Section S6 below) the counterbalanced execution order.

Table S5: End-to-end latency by committed transaction class.

Operation	n	Mean	SD	P95	P99
CheckAccess (pooled, committed)	62600	280.3	39.9	345.8	—
WriteSensor (committed)	146400	1219.8	60.0	1318.6	1358.9

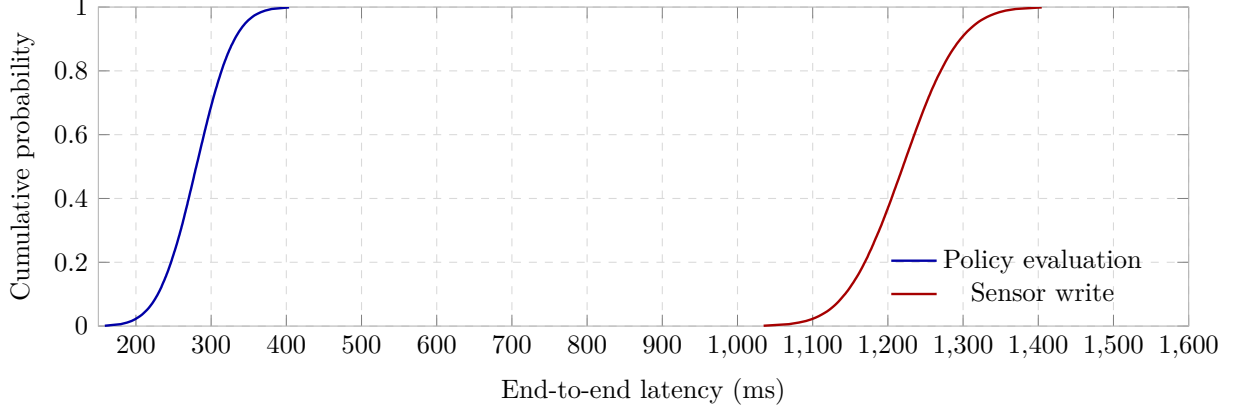


Figure S3: Empirical latency distributions for the two committed transaction classes.

S6 Additional results detail

This section gives the tables and figures behind the main article’s Results section that are not needed to follow the headline findings: the operation-class latency breakdown, the full throughput and peer-scaling latency tables (the main article keeps the figures), the peer-scaling execution order, the revocation lifecycle breakdown, the CRT residue recovery bounds, and the authorization-boundary denial-mechanism breakdown.

Table S6: Throughput by offered concurrency across five independent runs per cell.

Clients	Baseline	HRBAC	Reduction	p	p_{Holm}
10	44.5 ± 2.0	40.1 ± 1.2	10.0%	0.005	0.011
20	62.7 ± 3.4	55.0 ± 2.0	12.4%	0.004	0.011
30	67.4 ± 0.4	62.0 ± 1.0	7.9%	<0.001	<0.001
40	70.4 ± 2.1	63.5 ± 0.7	9.8%	0.001	0.005
50	69.1 ± 1.6	63.1 ± 0.7	8.6%	<0.001	0.002
60	69.3 ± 1.6	61.0 ± 0.9	12.0%	<0.001	<0.001
70	66.1 ± 0.9	59.8 ± 1.6	9.5%	<0.001	0.002
80	64.7 ± 1.9	59.7 ± 1.5	7.9%	0.002	0.008
90	61.3 ± 2.7	56.0 ± 0.9	8.6%	0.009	0.011
100	59.0 ± 0.1	53.8 ± 0.9	8.9%	<0.001	0.002

Table S7: Write latency across configured logical peer counts at 50 concurrent clients.

Peers	Runs	n	Mean (ms)	P95 (ms)	P95 95% CI
4	8	1040	1204.8	1296.8	[1287.4, 1309.0]
8	8	1040	1149.5	1249.1	[1224.0, 1263.0]
16	8	1040	1056.9	1151.2	[1135.6, 1166.9]
32	8	1040	811.8	906.4	[893.3, 920.7]

Table S8: Counterbalanced execution order for the eight peer-scaling blocks.

Block	Peer counts tested, in order
1	4, 8, 16, 32
2	8, 16, 32, 4
3	16, 32, 4, 8
4	32, 4, 8, 16
5	32, 16, 8, 4
6	4, 32, 16, 8
7	8, 4, 32, 16
8	16, 8, 4, 32

Table S9: Run-level mean latency and authorization-associated span across the peer-scaling campaign.

Seq.	Block	Peers	Run ID	Mean total (ms)	Mean span (ms)
1	1	4	PSR-01	1188.0	323.6
2	1	8	PSR-02	1156.9	315.4
3	1	16	PSR-03	1034.7	327.6
4	1	32	PSR-04	809.1	312.2
5	2	8	PSR-05	1144.5	328.0
6	2	16	PSR-06	1054.3	324.4
7	2	32	PSR-07	824.3	312.4
8	2	4	PSR-08	1208.0	321.8
9	3	16	PSR-09	1042.0	327.0
10	3	32	PSR-10	781.2	319.2
11	3	4	PSR-11	1189.8	313.1
12	3	8	PSR-12	1149.9	324.4
13	4	32	PSR-13	821.3	315.8
14	4	4	PSR-14	1202.6	323.9
15	4	8	PSR-15	1146.2	323.0
16	4	16	PSR-16	1085.2	316.6
17	5	32	PSR-17	789.9	320.1
18	5	16	PSR-18	1036.9	327.1
19	5	8	PSR-19	1143.0	320.5
20	5	4	PSR-20	1202.9	317.7
21	6	4	PSR-21	1206.2	313.7
22	6	32	PSR-22	830.8	309.7
23	6	16	PSR-23	1084.5	314.1
24	6	8	PSR-24	1131.7	317.8
25	7	8	PSR-25	1133.1	312.1
26	7	4	PSR-26	1212.5	328.6
27	7	32	PSR-27	799.9	321.8
28	7	16	PSR-28	1040.8	322.2
29	8	16	PSR-29	1076.7	324.2
30	8	8	PSR-30	1190.4	322.1
31	8	4	PSR-31	1228.1	330.0
32	8	32	PSR-32	837.9	322.9

Table S10: Revocation lifecycle records and delayed-stage rates by event type.

Event type	n	Delayed	Rate	Gossip (min)
CACHE_INVALIDATE	55	17	30.9%	4.24
CERT_REVOKE	53	13	24.5%	4.15
CRL_PUBLISH	46	18	39.1%	4.24
GOSSIP_COMPLETE	46	11	23.9%	4.22

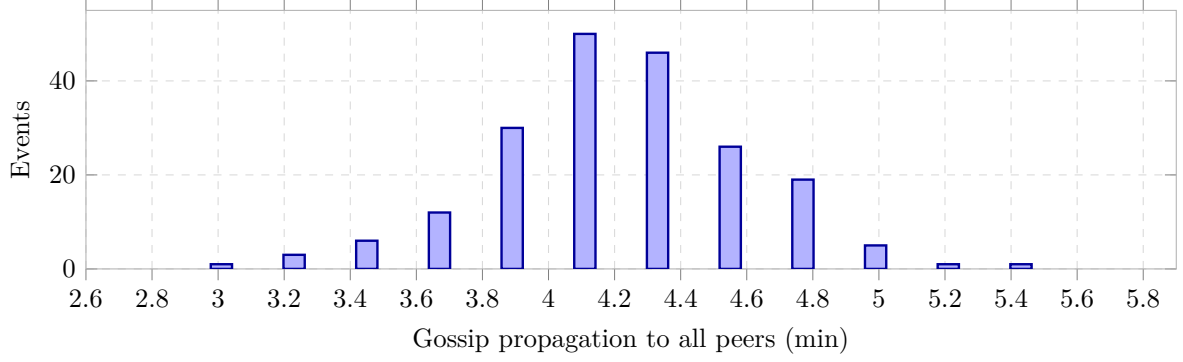


Figure S4: Gossip propagation across 200 revocation-lifecycle events.

S6.1 Latency by operation class

S6.2 Throughput

S6.3 Peer-scaling latency and execution order

S6.4 Revocation lifecycle

S6.5 CRT residue recovery bounds

S6.6 Authorization-boundary denial mechanisms

Table S11: Pairwise recovery bounds for the deployed residue moduli.

Residues present (missing modulus)	Recovery bound
97, 101 (missing 103)	$97 \times 101 = 9\,797$
97, 103 (missing 101)	$97 \times 103 = 9\,991$
101, 103 (missing 97)	$101 \times 103 = 10\,403$

Table S12: Scripted authorization requests grouped by recorded denial mechanism.

Observed denial mechanism	Count	Roles	Zones	Identities
Policy violation	1416	6	4	467
Insufficient role	1353	6	4	464
Revoked certificate	1340	6	4	462
Operation not permitted	1313	6	4	463
Gateway rate limit	1304	6	4	456
Zone mismatch	1274	6	4	465
Total	8000			

S7 Complete main-article figure set

For convenient review, Figures S5 to S8 reproduce every figure from the main article. The editable diagrams and plot data remain in the artifact.

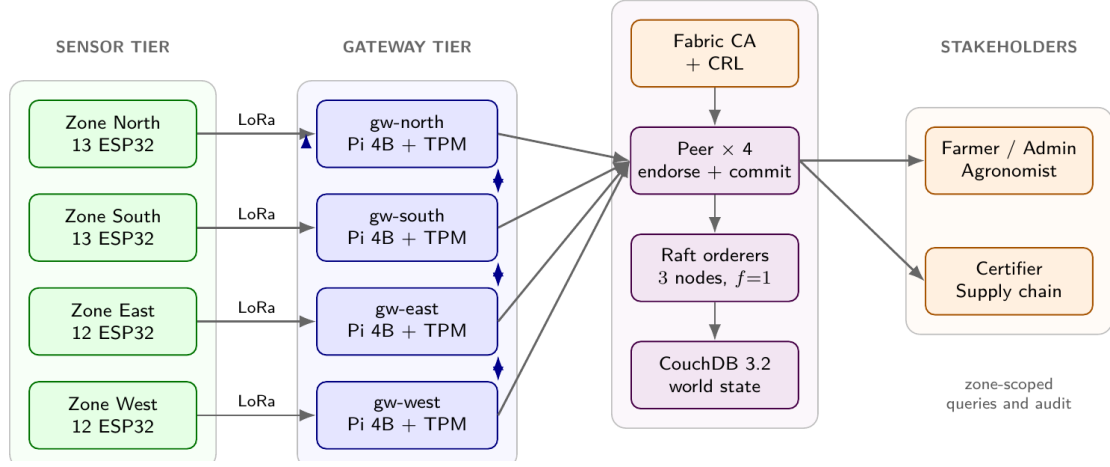


Figure S5: Deployment architecture linking sensors, zone gateways, Fabric services, and authorized users.

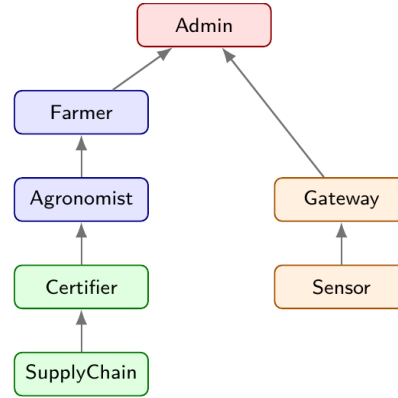


Figure S6: Seven-role hierarchy implemented in the deployed chaincode.

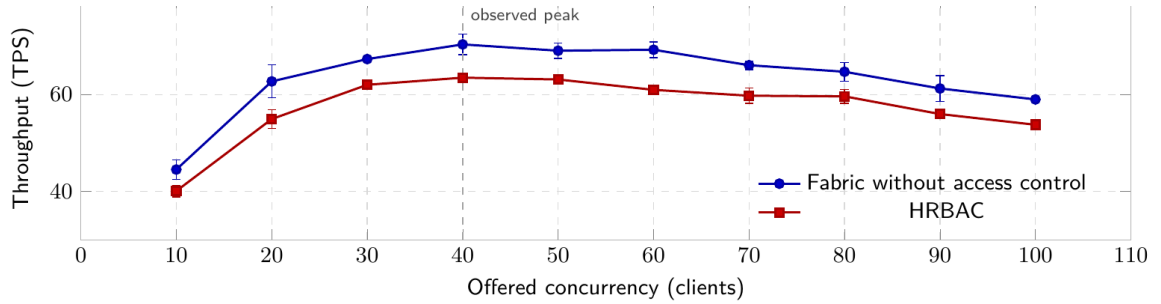


Figure S7: Mean throughput and standard deviation across five interleaved runs per concurrency level.

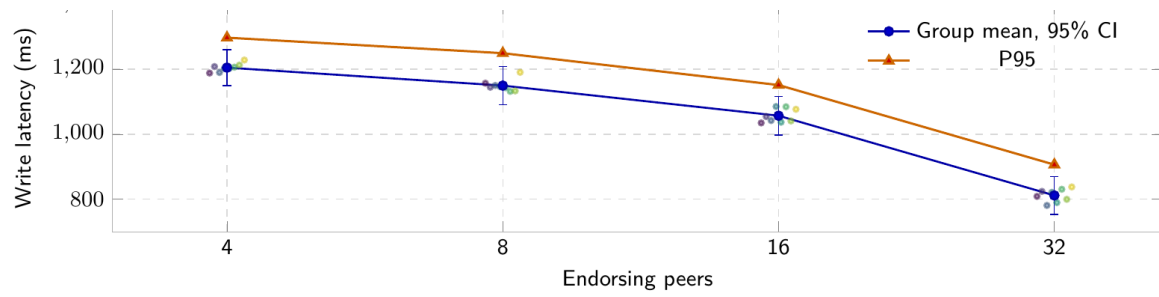


Figure S8: Run-level write latency and 95% confidence intervals by configured logical peer count.

References

- [1] Bandara, E., Liang, X., De Zoysa, K., Foytik, P., Shetty, S., 2021. Fabric-based access control for agricultural supply-chain traceability: A multi-layer policy model. *IEEE Transactions on Dependable and Secure Computing* 18, 2089–2101. doi:10.1109/TDSC.2021.3092089.
- [2] González, V., Sánchez, L., Lanza, J., Santana, J.R., Sotres, P., García, A.E., 2023. On the use of blockchain to enable a highly scalable Internet of Things data marketplace. *Internet of Things* 22, 100722. doi:10.1016/j.iot.2023.100722.
- [3] Idrissi, M., Palmieri, F., 2024. Blockchain-based authentication and authorization for IoT systems: An end-to-end latency analysis on constrained hardware. *The Journal of Supercomputing* 80, 1–32. doi:10.1007/s11227-024-06001-7.
- [4] Novo, O., 2018. Blockchain meets IoT: An architecture for scalable access management in IoT. *IEEE Internet of Things Journal* 5, 1184–1195. doi:10.1109/JIOT.2018.2812239.
- [5] Ouaddah, A., Abou Elkalam, A., Ait Ouahman, A., 2016. FairAccess: A new blockchain-based access control framework for the Internet of Things. *Security and Communication Networks* 9, 5943–5964. doi:10.1002/sec.1748.
- [6] Pajooh, H.H., Hammoudeh, M., Alam, F., Brocklehurst, S., 2021. Hyperledger fabric blockchain for securing the edge IoT: Systems architecture, experiments and critical observations. *Electronics* 10, 1338. doi:10.3390/electronics10111338.
- [7] Putra, G.D., Dedeoglu, V., Kanhere, S.S., Jurdak, R., 2021. Trust management in decentralized IoT access control system, in: *Proc. IEEE BCCA*, pp. 1–8. doi:10.1109/BCCA53669.2021.9657049.
- [8] Shih, D.H., Jheng, P.Y., Yen, D.C., Huang, C.L., 2022. A blockchain-based access control model for industrial IoT. *Sensors* 22, 2188. doi:10.3390/s22062188.
- [9] Usman, M., Jolfaei, A., Tariq, M.U., 2024. Blockchain-based access control for industrial IoT: Domain policy architecture with smart contracts. *Computer Networks* 243, 110286. doi:10.1016/j.comnet.2024.110286.
- [10] Xu, R., Chen, Y., Blasch, E., Chen, G., 2018. BlendCAC: A blockchain-enabled decentralized capability-based access control for IoTs, in: *Proceedings of the 2018 IEEE International Conference on Internet of Things (iThings)*, pp. 1027–1034. doi:10.1109/Cybermatics_2018.2018.00191.
- [11] Zaidi, S.M.S., Al-Qurishi, M., Abidin, M.Z.u., Almakdi, S., Asiri, Y., 2023. Fabrication: A hyperledger fabric framework for scalable IoT access control. *IEEE Internet of Things Journal* 10, 12636–12649. doi:10.1109/JIOT.2023.3254978.